

MACHINE LEARNING PROJECT REPORT

Ordinal Sentiment Analysis of Amazon Customer Reviews

Predicting 1–5 Star Ratings from Review Text Using TF-IDF and Ordinal Ridge Regression

Prepared by

Aleeza Nadeem

Computer Science (Machine Learning) ·

August 2026

Contents

01	Executive Summary.....	3
02	Problem Statement.....	4
03	Dataset Overview.....	5
04	Data Cleaning & Preprocessing.....	6
05	Exploratory Data Analysis.....	8
06	Feature Engineering.....	11
07	Model Architecture.....	12
08	Results & Evaluation.....	13
09	Business Insights & Discussion.....	15
10	Limitations & Future Work.....	16
11	Conclusion.....	17

01 Executive Summary

This report documents the design, implementation, and evaluation of an ordinal sentiment analysis model that predicts Amazon product review star ratings (1 to 5) directly from customer-written review text. Rather than treating the task as an unordered multi-class classification problem, the model exploits the natural ordering of star ratings — a 1-star and 5-star review are far more different than a 4-star and 5-star review — by framing prediction as a bounded regression problem solved with Ridge Regression over TF-IDF text features.

The model was trained and evaluated on roughly 99,000 cleaned Amazon Fine Food Reviews after deduplication and outlier filtering. It achieves a Mean Absolute Error of 0.57 stars and a Quadratic Weighted Kappa of 0.70, meaning predictions are, on average, within roughly half a star of the true rating and show strong ordinal agreement with human ratings — most errors fall on adjacent rating classes rather than being wildly off.

0.573**MAE**

Mean Absolute Error (stars)

0.703**QWK**

Quadratic Weighted Kappa

98,961**Reviews Modeled**

After cleaning & filtering

Key findings from the exploratory analysis show a strongly imbalanced rating distribution (62.6% of reviews are 5-star), a positive relationship between helpfulness votes and rating, and moderate variation in review length across rating classes — all of which informed preprocessing and evaluation choices described in this report.

02 Problem Statement

The Business Problem

E-commerce platforms such as Amazon collect millions of free-text product reviews every year. Star ratings summarize customer sentiment numerically, but the underlying review text carries far richer signal — specific complaints, praise, and context that a single number cannot capture. At the same time, many reviews are inconsistently rated relative to their text (a frustrated 3-star review can read more negatively than its score suggests), and a large share of user-generated content on the web lacks any rating at all (forum posts, support tickets, social mentions of a product).

A model that can reliably infer the star rating a piece of text implies has direct commercial value: it lets a business score unrated or under-rated feedback at scale, flag mismatches between text sentiment and posted rating for moderation, and monitor sentiment trends in near real time without waiting on manual tagging.

Why This Is an Ordinal Problem, Not a Standard Classification Problem

Star ratings are ordinal categories: $1 < 2 < 3 < 4 < 5$, with a meaningful notion of distance between classes. A conventional multi-class classifier (e.g. multinomial logistic regression or a softmax neural network) treats all misclassifications equally — confusing a 1-star review for a 5-star review is penalized the same as confusing it for a 2-star review, which does not reflect how mistaken the prediction actually is. This project instead treats the rating as a continuous quantity between 1 and 5, predicts it with regression, and rounds to the nearest whole star — a lightweight way to respect the ordering without the added complexity of a full ordinal-logit or proportional-odds model.

Problem this project solves

Given only the free text of a product review, predict the 1–5 star rating the reviewer assigned — in a way that penalizes predictions in proportion to how far off they are, and that generalizes to text without an existing rating.

03 Dataset Overview

The project uses the Amazon Fine Food Reviews dataset (Reviews.csv), a widely used public benchmark of customer reviews for food products sold on Amazon. Each row represents a single review, along with associated metadata about the product, reviewer, and community feedback on the review itself.

Column Reference

Column	Description	Role in Model
--------	-------------	---------------

Score	Star rating given by the reviewer (1–5)	Target variable
Summary	Short review headline	Combined into text feature
Text	Full body of the customer review	Primary text feature
HelpfulnessNumerator	Count of users who voted the review helpful	EDA / context only
HelpfulnessDenominator	Total votes cast (helpful + not helpful)	EDA / context only
Time	Unix timestamp of the review post	Converted to datetime for trend analysis
Id, ProductId, UserId, ProfileName	Identifiers and display name	Dropped — not predictive

Identifier columns (Id, ProductId, UserId, ProfileName) were dropped early since they carry no generalizable signal for predicting a rating from text. Score is preserved as the target; Summary and Text are concatenated into a single field so the model can learn from both the headline and the full review body.

04 Data Cleaning & Preprocessing

4.1 — Structural Cleaning

The raw file was loaded with malformed rows skipped (`on_bad_lines='skip'`) to tolerate any encoding irregularities in the source CSV. The four identifier columns were dropped, and the Unix Time column was converted to a proper datetime type to support any time-based analysis.

- Rows with a missing Score (the target) were removed outright — a review that cannot be labeled cannot train or evaluate the model.
- Exact duplicate reviews — identical Text posted at the identical Time — were removed, keeping only the first occurrence.

Removing duplicate text/time pairs matters for two reasons: it prevents repeated or spam-like reviews from over-representing certain words in the TF-IDF vocabulary, and it avoids the same review appearing in both the training and test splits, which would leak information and inflate the reported accuracy.

4.2 — Text Normalization

Summary and Text were concatenated into a single `full_text` field so the model sees both the headline and the full review body. A regex-based cleaning function then:

- Strips HTML artifacts such as `<br />` tags that are common in scraped review text.
- Removes punctuation, digits, and special characters, keeping only letters and spaces.
- Lowercases and trims the result for consistent tokenization downstream.

Rows with a missing Text were dropped (a review needs a body to model), while missing Summary values were filled with an empty string rather than dropped, since the review body alone is still usable.

4.3 — Outlier Handling by Review Length

Word count was computed for every cleaned review to understand the length distribution and identify reviews too short to carry sentiment signal or too long to be a typical review (likely bulk-pasted or spam content).

Statistic	Value
Count	99,959 reviews (pre-length-filter)
Mean word count	83.4 words
Median word count	61 words
Std. deviation	75.8 words
1st percentile	19 words
99th percentile	374 words

Maximum	2,369 words
---------	-------------

Reviews below 2 words were removed as carrying insufficient sentiment signal, and the top 1% by length (above 374 words) was trimmed as likely spam or outlier content. This brought the dataset from 99,959 to 98,961 rows — a removal of 998 reviews (about 1.0% of the pre-filter data), a controlled trim that preserves the vast majority of the corpus while removing its noisiest tails.

05 Exploratory Data Analysis

With the dataset cleaned, three questions were explored before modeling: how imbalanced are the star ratings, does review length vary meaningfully with rating, and does the community's helpfulness voting relate to how positive or negative a review is.

5.1 — Class Imbalance in Star Ratings

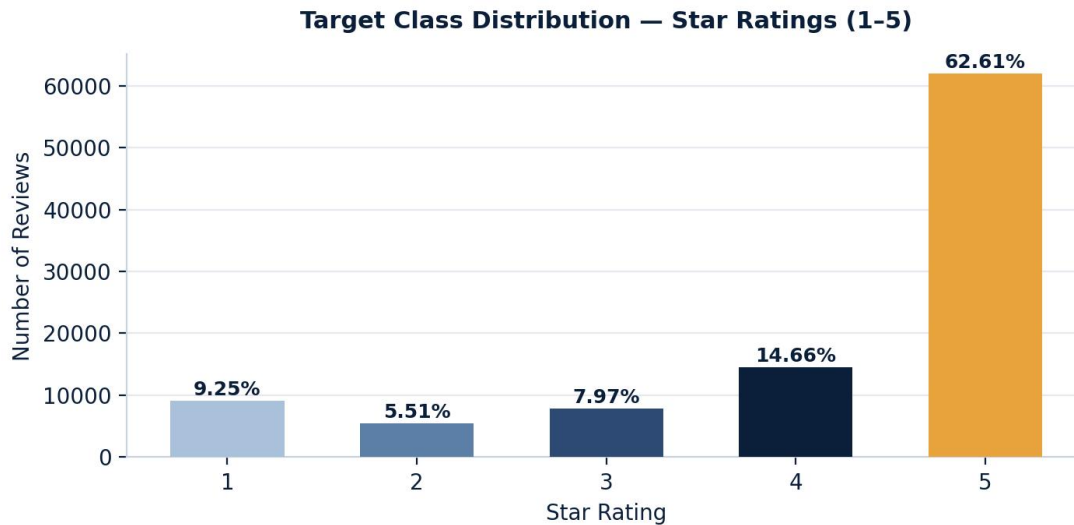


Figure 1 — Distribution of star ratings across the cleaned dataset ($n = 98,961$).

The rating distribution is heavily skewed toward positive reviews: 62.6% of all reviews are 5-star, and 4-star reviews add another 14.7% — meaning over three-quarters of the dataset is positively rated. The three lower ratings are comparatively rare, with 1-star at 9.3%, 3-star at 8.0%, and 2-star the smallest at 5.5%.

Modeling implication

This class imbalance is the reason the model is evaluated with Mean Absolute Error and Quadratic Weighted Kappa rather than plain accuracy — a model that always predicts "5" would score misleadingly well on accuracy alone despite having no real discriminative power for negative reviews.

5.2 — Review Length Across Ratings

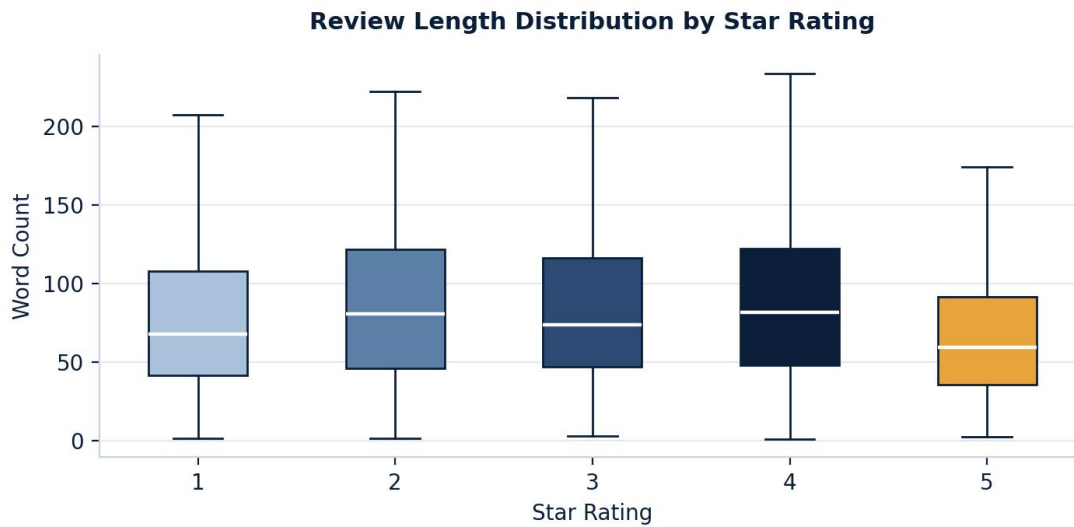


Figure 2 — Word count distribution by star rating (outliers beyond the 99th percentile trimmed).

Star Rating	Mean Words	Median Words	Std. Dev.
1	82.0	64	59.8
2	90.2	73	62.5
3	95.8	76	67.6
4	90.4	70	66.6
5	72.8	55	55.7

Mid-range ratings (2, 3, and 4 stars) tend to be the longest — reviewers with a mixed or moderate experience apparently feel the need to explain themselves in more detail, listing both pros and cons. 5-star reviews are the shortest on average (73 words), suggesting satisfied customers often leave brief, enthusiastic feedback rather than a detailed account. Because raw word count varies with rating, the TF-IDF feature extraction step (Section 6) applies L2 normalization so that longer reviews are not artificially weighted as more "neutral" simply due to their length.

5.3 — Helpfulness Ratio vs. Rating

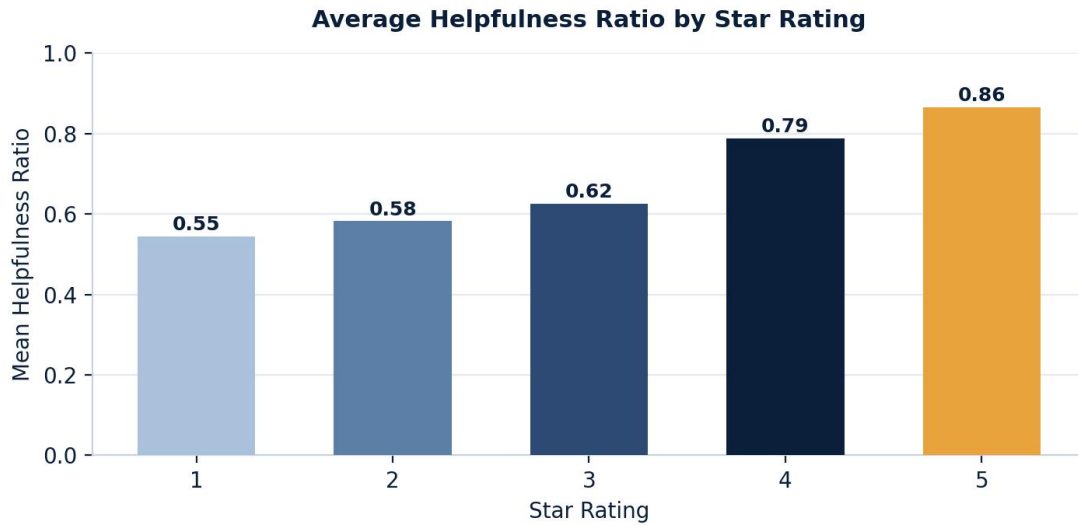


Figure 3 — Mean helpfulness ratio ($\text{upvotes} \div \text{total votes}$) by star rating, restricted to reviews that received at least one vote.

There is a clear upward trend: reviews with higher star ratings tend to be voted more "helpful" by other users, rising from a mean ratio of about 0.55 for 1-star reviews to about 0.86 for 5-star reviews. This suggests community helpfulness voting is not neutral with respect to sentiment — other shoppers are more likely to endorse positive reviews as useful, which is a useful piece of context when interpreting helpfulness as a feature in future extensions of this model but was not used as a direct model input here.

06 Feature Engineering

6.1 — Train/Test Split

The cleaned dataset was split 80/20 into training and test sets, stratified on Score so that the proportion of each star rating is preserved identically in both partitions — important given the class imbalance documented in Section 5.1.

6.2 — TF-IDF Vectorization

Review text was converted into numeric features using Term Frequency–Inverse Document Frequency (TF-IDF) vectorization, configured as follows:

Parameter	Setting	Purpose
stop_words	'english'	Removes uninformative high-frequency words ("the", "and", ...)
max_features	5,000	Caps vocabulary size to the 5,000 most informative terms
ngram_range	(1, 2)	Captures both single words and two-word phrases (e.g. "not bad")

Including bigrams alongside single words (unigrams) matters for sentiment: a phrase like "not bad" or "not good" flips the meaning of the individual words it contains, and a unigram-only model would miss that negation entirely.

The vectorizer was fit exclusively on the training set and then used to transform the test set — never the reverse. Fitting on the full dataset before splitting would leak vocabulary and document-frequency statistics from the test set into training, producing an overly optimistic evaluation that would not hold up on genuinely unseen reviews.

07 Model Architecture

7.1 — Ordinal Regression via Ridge

Rather than a standard multi-class classifier, the model uses Ridge Regression (L2-regularized linear regression, $\alpha = 1.0$) to predict the star rating as a continuous value between 1 and 5. This directly encodes the ordinal structure of the target: the model is optimized to minimize squared error against the true numeric rating, so a prediction of 4.8 for a true 5-star review is penalized far less than a prediction of 1.2 would be — behavior a standard classifier's cross-entropy loss does not naturally provide.

7.2 — Post-Processing

The raw regression output is a continuous number that can technically fall outside the valid 1–5 range (e.g. 5.3 or 0.7). Two post-processing steps convert this into a valid, interpretable star rating:

- Clipping — predictions are clipped to the $[1, 5]$ bound using `np.clip`, so no prediction can fall outside the valid rating scale.
- Rounding — the clipped value is rounded to the nearest integer star rating, since Amazon ratings are always whole numbers.

Why Ridge over a plain classifier

A multinomial classifier (e.g. softmax logistic regression) would need five independent decision boundaries and treats a 1-vs-5 confusion the same as a 4-vs-5 confusion. Framing this as bounded regression is a lightweight way to bake in the ordinal relationship between classes without the added modeling complexity of a full proportional-odds or ordinal-logit architecture.

7.3 — Pipeline Summary

- Input: cleaned review text (Summary + Text, normalized)
- Vectorization: TF-IDF, 5,000 features, unigrams + bigrams, English stop words removed
- Model: Ridge Regression ($\alpha = 1.0$), trained to predict Score as a continuous target
- Output: raw prediction $\rightarrow$ clipped to $[1, 5]$ $\rightarrow$ rounded to nearest integer star rating

08 Results & Evaluation

Because standard classification accuracy does not reflect how close a prediction is to the true rating — and would be distorted by the class imbalance seen in Section 5.1 — the model is evaluated with two ordinal-aware metrics.

Mean Absolute Error

0.573

Average star-rating deviation

Quadratic Weighted Kappa

0.703

Ordinal agreement with true ratings

8.1 — Interpreting the Metrics

Mean Absolute Error (MAE) of 0.573 means predictions are, on average, just over half a star away from the reviewer's actual rating — for a 1–5 scale, this is a tight margin. Quadratic Weighted Kappa (QWK) of 0.703 measures agreement between predicted and actual ratings while weighting larger disagreements more heavily than smaller ones; a value in this range is generally considered strong-to-substantial agreement, well above the 0 expected from random guessing and the roughly 0.4–0.6 range typical of moderate agreement.

8.2 — Confusion Matrix

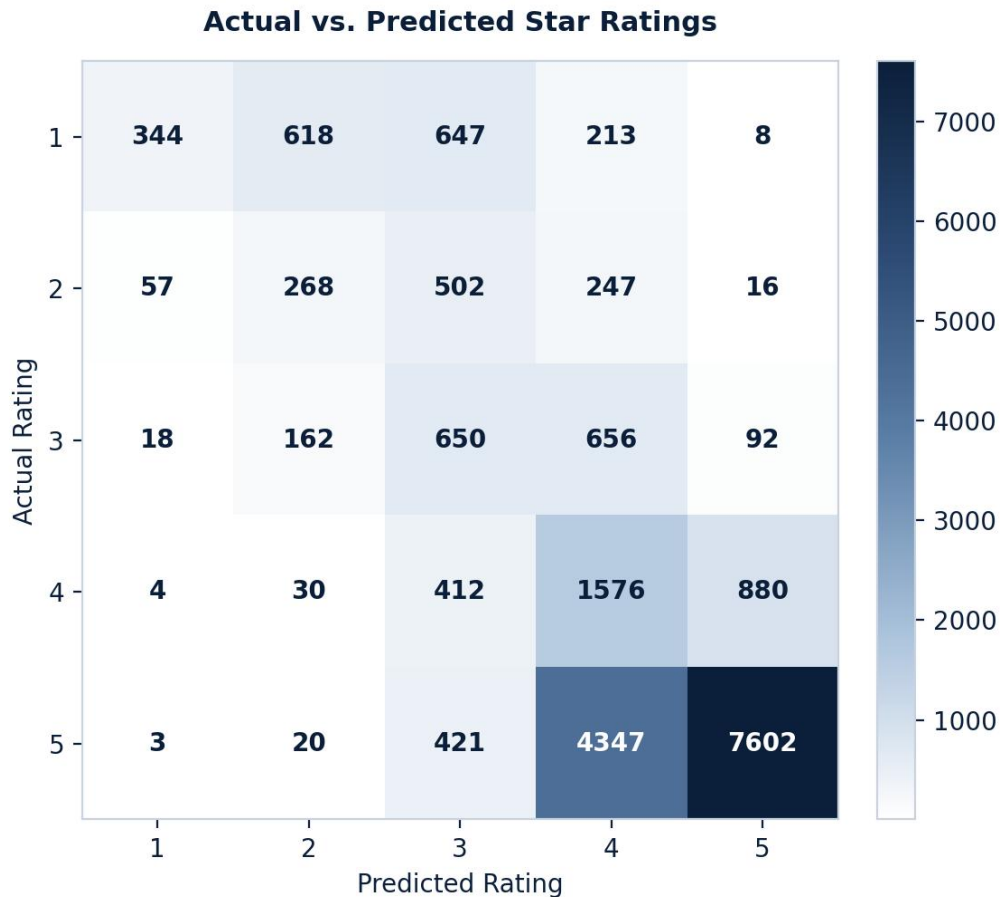


Figure 4 — Actual vs. predicted star ratings on the held-out test set.

The confusion matrix confirms the ordinal behavior the model was designed for: errors concentrate along the diagonal and its immediate neighbors rather than scattering across the full scale. Only 8 true 1-star reviews were predicted as 5-star (out of 1,830 total 1-star reviews), and only 3 true 5-star reviews were predicted as 1-star — the model very rarely makes a "maximally wrong" prediction.

The clearest weakness is in the middle of the scale: true 2-star and 3-star reviews are frequently predicted as 3 or 4 stars rather than their exact class (e.g. 502 true 2-star reviews predicted as 3-star). This mirrors the class imbalance from Section 5.1 — with comparatively few 2- and 3-star training examples, the model gravitates toward the much more common 4- and 5-star predictions when review text is ambiguous or moderately toned.

09 Business Insights & Discussion

- Text alone predicts rating with useful precision. A purely text-based model, without any product or reviewer metadata, gets within roughly half a star of the true rating on average — supporting the use case of scoring unrated or under-rated text (support tickets, social mentions, forum posts) using the same approach.
- Sentiment is rarely mistaken for its opposite. The near-absence of 1-star-predicted-as-5-star (and vice versa) errors means this model is well suited to flagging genuinely mismatched review/rating pairs for moderation, since large prediction-vs-actual gaps are a meaningfully rare and therefore high-signal event.
- The middle of the scale is where the model needs the most help. 2- and 3-star reviews are the hardest to separate from 4-star reviews, likely because moderately-toned language ('it's okay,' 'decent, but...') is intrinsically more ambiguous than clearly positive or negative language — and because the model has the least training data for these classes.
- Helpfulness voting tracks sentiment. Since helpful votes rise steadily with rating (Section 5.3), helpfulness signals could enrich future models — though they were deliberately excluded here since they are not available at write-time for a brand-new, unvoted review.

10 Limitations & Future Work

Limitations

- Class imbalance. With only 5.5% of reviews at 2-star and 8.0% at 3-star, the model has comparatively little signal to learn these classes well, contributing directly to the mid-scale confusion in Section 8.2.
- Linear model ceiling. Ridge Regression over TF-IDF cannot capture word order, sarcasm, or long-range context ("the box looked great but the product itself was a huge disappointment") the way a sequence model or transformer-based embedding could.
- Single-domain data. The Amazon Fine Food Reviews dataset is food-product-specific; vocabulary and sentiment patterns may not transfer cleanly to other product categories without retraining.

Future Work

- Address class imbalance directly with class-weighted loss, oversampling of minority rating classes (e.g. SMOTE-style techniques adapted for text), or a two-stage model that first separates extreme from moderate ratings.
- Replace TF-IDF with contextual embeddings (e.g. a fine-tuned transformer) to capture negation, sarcasm, and long-range dependencies that bag-of-words features miss.
- Explore a true ordinal model — proportional-odds logistic regression or an ordinal neural loss (e.g. CORAL/CORN) — as a more principled alternative to the clip-and-round Ridge approach used here.
- Incorporate the helpfulness ratio as a post-hoc confidence or moderation signal for already-rated reviews, separate from the core rating-prediction task.

11 Conclusion

This project set out to answer a practical question: can a business reliably infer a customer's star rating from review text alone, and do so in a way that respects how ordered and continuous that rating scale really is? The answer, on this dataset, is yes — a comparatively simple TF-IDF plus Ridge Regression pipeline reaches a Mean Absolute Error of 0.573 stars and a Quadratic Weighted Kappa of 0.703, with prediction errors concentrated among adjacent rating classes rather than scattered across the full scale.

The approach demonstrates that framing star-rating prediction as bounded ordinal regression, rather than standard multi-class classification, is a lightweight but effective way to encode the natural ordering of the target — without the added architectural complexity of a dedicated ordinal model. The clearest opportunity for improvement lies in the middle of the rating scale, where class imbalance and linguistically ambiguous moderate reviews remain the model's primary source of error, pointing directly to the future work outlined in Section 10.